

SimBench

A Portable Benchmarking Methodology for Full-System Simulators

Harry Wagstaff Bruno Bodin Tom Spink Björn Franke

Institute for Computing Systems Architecture
University of Edinburgh

ISPASS 2017

Motivation

- Instruction Set Simulation
- Evaluating Simulation Tools

The SimBench Methodology

- Overview
- Implementation
- Porting SimBench

Evaluation

Instruction Set Simulation

Instruction Set Simulation is used in a wide variety of contexts:

Instruction Set Simulation

Instruction Set Simulation is used in a wide variety of contexts:

- Design Space Exploration
 - Gem5
 - Multi2Sim



Instruction Set Simulation

Instruction Set Simulation is used in a wide variety of contexts:

- Design Space Exploration
- Software Development
 - QEMU
 - Android Emulator



Instruction Set Simulation

Instruction Set Simulation is used in a wide variety of contexts:

- Design Space Exploration
- Software Development
- Backwards Compatibility
 - Apple Rosetta
 - Nintendo NES Classic



Instruction Set Simulation

Instruction Set Simulation is used in a wide variety of contexts:

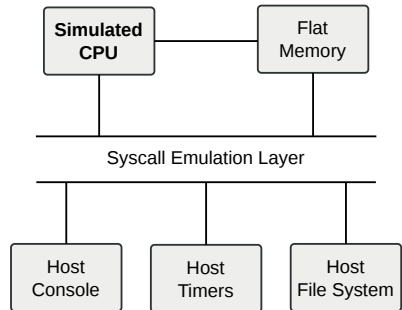
- Design Space Exploration
- Software Development
- Backwards Compatibility

Simulators can be broken up into several categories:

- Functional Only
- Profiling/Performance Modelling
- User-Mode
- Full System

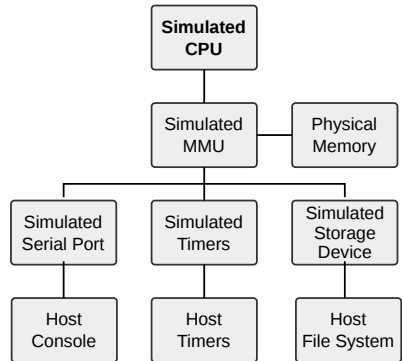
User-Mode Simulation

- Execute a single binary
- Emulate System Calls
- Simplified Memory System



Full-System Simulation

- Boot out of reset
- System-mode Instructions
- Memory Translation
- Asynchronous Interrupts
- External Devices



Fast Simulation

We want to make our simulators go fast!

To be clear, we want the simulator itself to be efficient.

Fast Simulation

We want to make our simulators go fast!

To be clear, we want the simulator itself to be efficient.

How?

- Dynamic Binary Translation
- Memory-Related Techniques
- Efficient Interrupt Modelling
- Control Flow Handling

Fast Simulation

We want to make our simulators go fast!

To be clear, we want the simulator itself to be efficient.

How?

- Dynamic Binary Translation
- Memory-Related Techniques
- Efficient Interrupt Modelling
- Control Flow Handling

How do we evaluate the effectiveness of these techniques in spite of complex interactions?

Typical Benchmarking Approaches

Most common approaches fall into two categories:

- Large, complex macrobenchmark suites
- Small, targeted but ad-hoc microbenchmarks

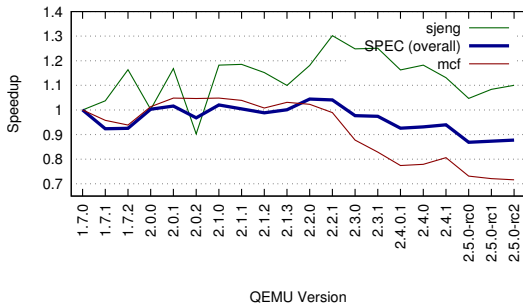
Both approaches are problematic for several reasons.

Macrobenchmarking

The most common approach is to use large macrobenchmarks

However, this presents several problems:

- Benchmark runtimes are long
- Difficult to perform detailed analysis
- Aggregated runtimes may hide interesting results



Ad-Hoc Microbenchmarking

Many papers also attempt to use ad-hoc microbenchmarks to assess performance.

However, these also have problems:

- Source code may be unavailable, damaging reproducibility
- Secondary effects may not be taken into account

Sum Two Arrays - Source Code

```
float out[ARRAY_SIZE];  
float a[ARRAY_SIZE];  
float b[ARRAY_SIZE];  
...  
void foo() {  
    for(int i = 0; i < ARRAY_SIZE; ++i) {  
        out[i] = a[i] + b[i];  
    }  
}
```

Sum Two Arrays - Assembly Code

```
foo:
    0:  add r3, r0, #512 ; 0x200
    4:  vldmia r0!, {s15}
    8:  vldmia r1!, {s14}
    c:  cmp r0, r3
   10:  vadd.f32 s15, s15, s14
   14:  vstmia r2!, {s15}
   18:  bne 4 <foo+0x4>
  1c:  bx lr
```

Sum Two Arrays - Assembly Code

```
foo:
    0:  add r3, r0, #512 ; 0x200
    4:  vldmia r0!, {s15}
    8:  vldmia r1!, {s14}
    c:  cmp r0, r3
   10:  vadd.f32 s15, s15, s14
   14:  vstmia r2!, {s15}
   18:  bne 4 <foo+0x4>
  1c:  bx lr
```

- ✗ Instruction Page Fault
- ✗ Code Generation

Instruction Page fault, Code generation

Sum Two Arrays - Assembly Code

```
foo:
  0:  add r3, r0, #512 ; 0x200
  4:  vldmia r0!, {s15}
  8:  vldmia r1!, {s14}
  c:  cmp r0, r3
 10:  vadd.f32 s15, s15, s14
 14:  vstmia r2!, {s15}
 18:  bne 4 <foo+0x4>
 1c:  bx lr
```

- x** Instruction Page Fault
- x** Code Generation
- x** Cold Memory Access
- x** Data Page Fault

Cold memory access, Data Page fault

Sum Two Arrays - Assembly Code

```
foo:
  0:  add r3, r0, #512 ; 0x200
  4:  vldmia r0!, {s15}
  8:  vldmia r1!, {s14}
  c:  cmp r0, r3
 10:  vadd.f32 s15, s15, s14
 14:  vstmia r2!, {s15}
 18:  bne 4 <foo+0x4>
1c:  bx lr
```

- x** Instruction Page Fault
- x** Code Generation
- x** Cold Memory Access
- x** Data Page Fault
- x** Hot Memory Access

Hot memory access

Sum Two Arrays - Assembly Code

```
foo:
    0:  add r3, r0, #512 ; 0x200
    4:  vldmia r0!, {s15}
    8:  vldmia r1!, {s14}
    c:  cmp r0, r3
   10:  vadd.f32 s15, s15, s14
   14:  vstmia r2!, {s15}
   18:  bne 4 <foo+0x4>
  1c:  bx lr
```

- ✗ Instruction Page Fault
- ✗ Code Generation
- ✗ Cold Memory Access
- ✗ Data Page Fault
- ✗ Hot Memory Access
- ✓ FP Operation

FP Operation

Sum Two Arrays - Assembly Code

```
foo:
  0:  add r3, r0, #512 ; 0x200
  4:  vldmia r0!, {s15}
  8:  vldmia r1!, {s14}
  c:  cmp r0, r3
 10:  vadd.f32 s15, s15, s14
 14:  vstmia r2!, {s15}
 18:  bne 4 <foo+0x4>
 1c:  bx lr
```

- ✗ Instruction Page Fault
- ✗ Code Generation
- ✗ Cold Memory Access
- ✗ Data Page Fault
- ✗✗ Hot Memory Access
- ✓ FP Operation

Hot memory access

Sum Two Arrays - Assembly Code

```
foo:
  0:  add r3, r0, #512 ; 0x200
  4:  vldmia r0!, {s15}
  8:  vldmia r1!, {s14}
  c:  cmp r0, r3
 10:  vadd.f32 s15, s15, s14
 14:  vstmia r2!, {s15}
 18:  bne 4 <foo+0x4>
 1c:  bx lr
```

- ✗ Instruction Page Fault
- ✗ Code Generation
- ✗ Cold Memory Access
- ✗ Data Page Fault
- ✗✗ Hot Memory Access
- ✓ FP Operation
- ✗ Direct Control Flow
- ✗ Interrupt

Direct control flow, Interrupt

Sum Two Arrays - Assembly Code

```
foo:
    0:  add r3, r0, #512 ; 0x200
    4:  vldmia r0!, {s15}
    8:  vldmia r1!, {s14}
    c:  cmp r0, r3
   10:  vadd.f32 s15, s15, s14
   14:  vstmia r2!, {s15}
   18:  bne 4 <foo+0x4>
  1c:  bx lr
```

- x Instruction Page Fault
- x x Code Generation
- x Cold Memory Access
- x Data Page Fault
- x x Hot Memory Access
- ✓ FP Operation
- x Direct Control Flow
- x Interrupt

Code generation

Sum Two Arrays - Assembly Code

```
foo:
    0:  add r3, r0, #512 ; 0x200
    4:  vldmia r0!, {s15}
    8:  vldmia r1!, {s14}
    c:  cmp r0, r3
   10:  vadd.f32 s15, s15, s14
   14:  vstmia r2!, {s15}
   18:  bne 4 <foo+0x4>
1c:  bx lr
```

- x** Instruction Page Fault
- x x x** Code Generation
- x** Cold Memory Access
- x** Data Page Fault
- x x** Hot Memory Access
- ✓** FP Operation
- x** Direct Control Flow
- x** Interrupt

Code generation

Sum Two Arrays - Assembly Code

```
foo:
  0:  add r3, r0, #512 ; 0x200
  4:  vldmia r0!, {s15}
  8:  vldmia r1!, {s14}
  c:  cmp r0, r3
 10:  vadd.f32 s15, s15, s14
 14:  vstmia r2!, {s15}
 18:  bne 4 <foo+0x4>
1c:  bx lr
```

- x** Instruction Page Fault
- x x x** Code Generation
- x** Cold Memory Access
- x** Data Page Fault
- x x** Hot Memory Access
- ✓** FP Operation
- x** Direct Control Flow
- x x** Interrupt
- x** Indirect Control Flow

Indirect control flow, Interrupt

SimBench

We present SimBench, which is designed to address some of these problems.

SimBench

We present SimBench, which is designed to address some of these problems.

SimBench...

- **Contains a range of targeted microbenchmarks**

SimBench

We present SimBench, which is designed to address some of these problems.

SimBench. . .

- Contains a range of targeted microbenchmarks
- **Is aimed at full-system simulation techniques**

SimBench

We present SimBench, which is designed to address some of these problems.

SimBench...

- Contains a range of targeted microbenchmarks
- Is aimed at full-system simulation techniques
- **Is easily portable to new platforms and architectures**

SimBench

We present SimBench, which is designed to address some of these problems.

SimBench...

- Contains a range of targeted microbenchmarks
- Is aimed at full-system simulation techniques
- Is easily portable to new platforms and architectures
- **Runs 'bare-metal' on the guest (i.e., without an OS)**

Guest System

Benchmarks



Platform Library

Arch. Library

Timing Application

Host System

Guest System

Benchmarks



Platform Library

Arch. Library

Timing Application

Host System

Categories

SimBench currently features five categories of benchmark:

- Code Generation
- Control Flow
- Exception Handling
- I/O
- Memory System

A total of 18 benchmarks are in the suite.

Implementation

The benchmarks are implemented entirely in C:

- No inline assembly in the benchmarks
- No weird intrinsics or builtins
- Some extensions used for code alignment

Implementation - Defeating Optimisations

We want to preserve interesting operations while still using our compiler optimisations:

- **Use of *empty volatile assembly statements***
- Volatile memory accesses where necessary
- Indirect jumps via opaque pointers where necessary
- Benchmarks split into many objects to defeat inlining

```
for(int i = 0; i < ITERATIONS; ++i) {  
    volatile asm (" ::: memory);  
}
```

Implementation - Defeating Optimisations

We want to preserve interesting operations while still using our compiler optimisations:

- Use of *empty* volatile assembly statements
- **Volatile memory accesses where necessary**
- Indirect jumps via opaque pointers where necessary
- Benchmarks split into many objects to defeat inlining

```
static volatile uint32_t value = 0;
...
value = value;
value = value;
value = value;
```

Implementation - Defeating Optimisations

We want to preserve interesting operations while still using our compiler optimisations:

- Use of *empty* volatile assembly statements
- Volatile memory accesses where necessary
- **Indirect jumps via opaque pointers where necessary**
- Benchmarks split into many objects to defeat inlining

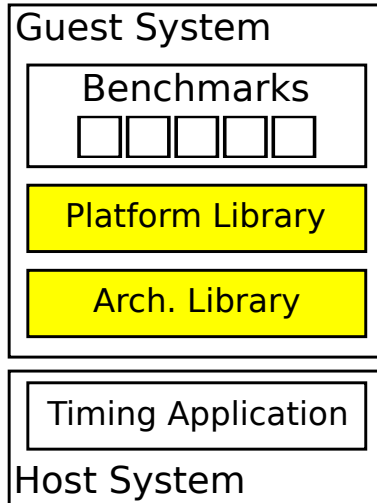
```
static benchmark_kernel_t fn_table[];  
...  
while(iterations-- > 0) {  
    fn_table[i % FN_TABLE_SIZE]();  
}
```

Implementation - Defeating Optimisations

We want to preserve interesting operations while still using our compiler optimisations:

- Use of *empty* volatile assembly statements
- Volatile memory accesses where necessary
- Indirect jumps via opaque pointers where necessary
- **Benchmarks split into many objects to defeat inlining**

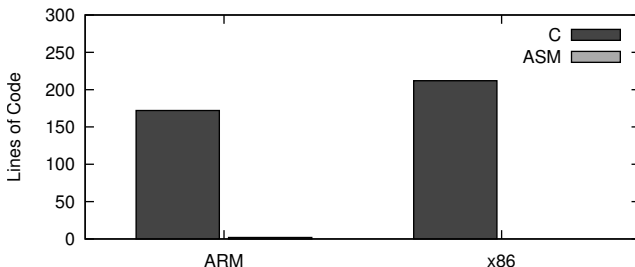
```
inter-page-direct.c:
    void ipd-fn-10();
    ...
    ipd-fn-10();
inter-page-direct-10.c:
    void ipd-fn-9();
    void ipd-fn-10() { ipd-fn-9(); }
```



Porting To A New Platform

To port SimBench to a new platform (where the architecture is already supported), 4 main components are required:

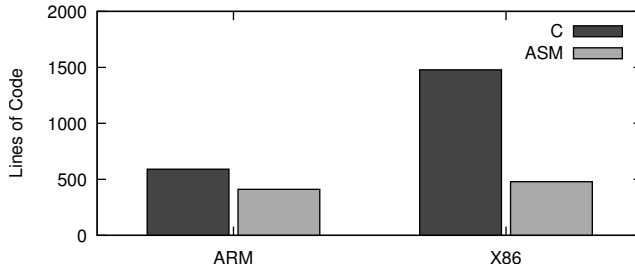
- A simple UART driver
- An IRQ Controller driver
- A description of the memory layout
- A linker script



Porting To A New Architecture

Porting to a new architecture is somewhat more complex.

- Bring system out of reset
- Manage page tables
- Manage interrupt/exception vectors



Guest System

Benchmarks



Platform Library

Arch. Library

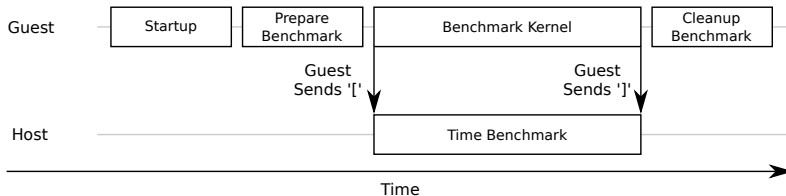
Timing Application

Host System

Timing

SimBench uses the Host system to collect benchmark timings:

- We don't trust the timing accuracy of the Guest
- Guest might report simulated cycles instead of real time
- Only needs platform-specific UART rather than Timer+IRQ+handler+...



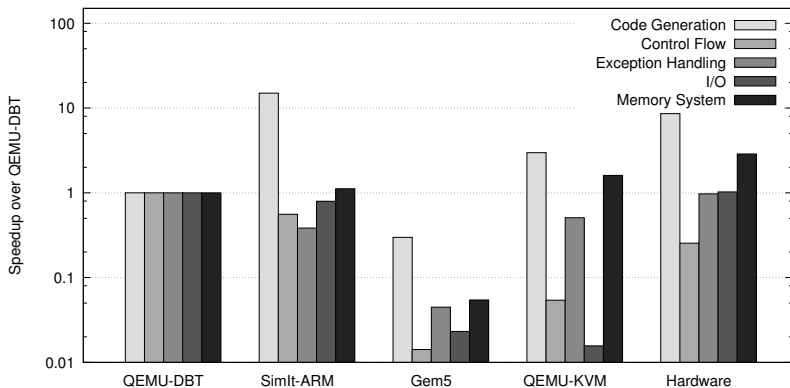
Evaluation

We ran SimBench on a variety of platforms

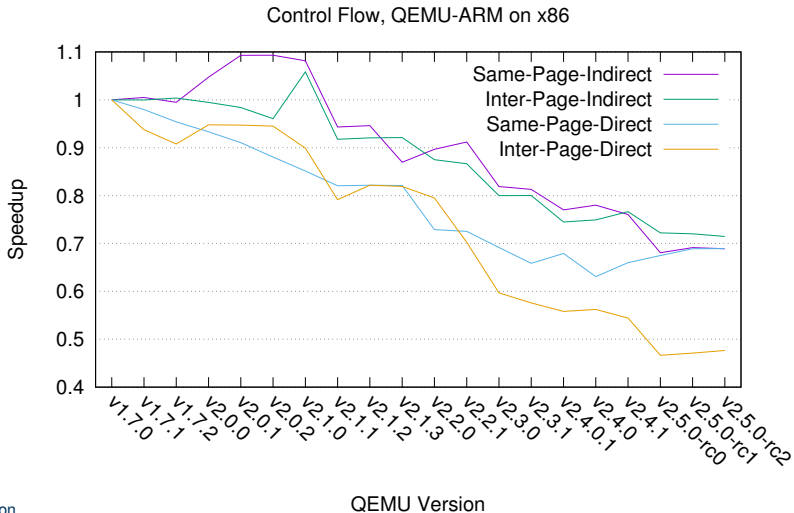
- A range of full system simulators including QEMU
- A virtualized system, using KVM
- Natively on real hardware

Categories - ARM Guest

Simulators on x86 Host, QEMU-KVM and Hardware on ARM Host

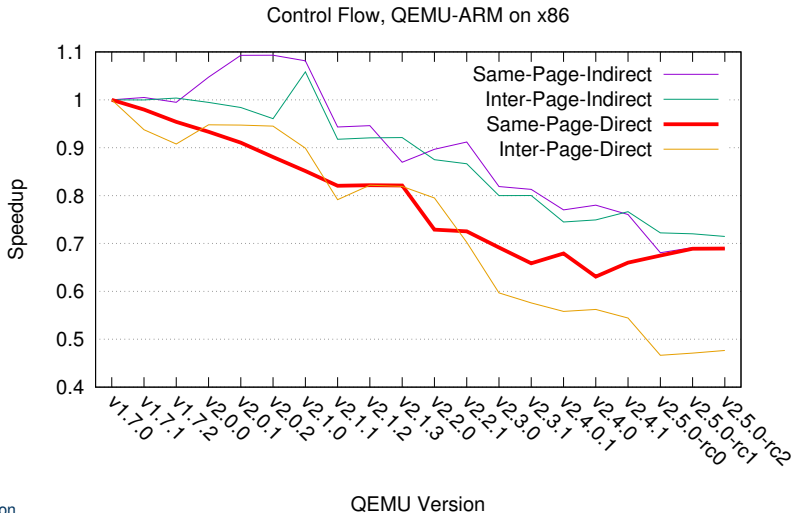


Categories



Categories

This represents 13% of executed instructions!



Summary

In summary:

- Existing benchmarking methods have several shortcomings
- SimBench is capable of addressing these shortcomings
- But SimBench does not replace application benchmarks

Future Work:

- Additional categories of benchmark
- Ports to new architectures & platforms
- Improve robustness of existing benchmarks

Thanks for Listening!
Any Questions?

SimBench is available now at
<http://bitbucket.org/simbench/simbench>

Benchmarks

- Code Generation
 - Small Blocks
 - Large Blocks
- Control Flow
 - Inter-Page Direct
 - Inter-Page Indirect
 - Intra-Page Direct
 - Intra-Page Indirect
- I/O
 - Memory Mapped Device
 - Coprocessor Access
- Exception Handling
 - Data Access Fault
 - Instruction Access Fault
 - Undefined Instruction
 - System Call
 - External Software Interrupt
- Memory
 - Cold Memory Access
 - Hot Memory Access
 - Nonprivileged Access
 - TLB Eviction
 - TLB Flush

Evaluation Platforms

Machine	ODROID-XU3	HP z440
CPU	Exynos 5422	Xeon E5-1620 v3
CPU GHz	2.0 (A15) 1.4 (A7)	3.5 (3.6 Boost)
Memory	2GB	16GB
Compiler	gcc 4.8.2	gcc 5.3.1
OS Name	Ubuntu 14.04	Fedora 21
OS Kernel	3.10.53	4.1.13